2023 IEEE International Conference on Robotics and Automation (ICRA 2023)
May 29 - June 2, 2023. London, UK

RAMP-Net: A Robust Adaptive MPC for Quadrotors via Physics-informed Neural Network

Sourav Sanyal (*Graduate Student Member, IEEE*) and Kaushik Roy (*Fellow, IEEE*)
Elmore Family School of Electrical and Computer Engineering, Purdue University
{sanyals, kaushik}@purdue.edu

2023 IEEE International Conference on Robotics and Automation (ICRA) | 979-8-3503-2365-8/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICRA48891.2023.10161410

***Abstract*—Model Predictive Control (MPC) is a state-of-the-art (SOTA) control technique which requires solving hard constrained optimization problems iteratively. For uncertain dynamics, analytical model based robust MPC imposes additional constraints, increasing the hardness of the problem. The problem exacerbates in performance-critical applications, when more compute is required in lesser time. Data-driven regression methods such as Neural Networks have been proposed in the past to approximate system dynamics. However, such models rely on high volumes of labeled data, in the absence of symbolic analytical priors. This incurs non-trivial training overheads. Physics-informed Neural Networks (PINNs) have gained traction for approximating non-linear system of ordinary differential equations (ODEs), with reasonable accuracy. In this work, we propose a Robust Adaptive MPC framework via PINNs (RAMP-Net), which uses a neural network trained partly from simple ODEs and partly from data. A physics loss is used to learn simple ODEs representing ideal dynamics. Having access to analytical functions inside the loss function acts as a regularizer, enforcing robust behavior for parametric uncertainties. On the other hand, a regular data loss is used for adapting to residual disturbances (non-parametric uncertainties), unaccounted during mathematical modelling. Experiments are performed in a simulated environment for trajectory tracking of a quadrotor. We report 7.8% to 43.2% and 8.04% to 61.5% reduction in tracking errors for speeds ranging from 0.5 to 1.75 m/s compared to two SOTA regression based MPC methods.**

I. Introduction

Model Predictive Control (MPC) [1] is an advanced control technique, which involves solving an optimal control problem iteratively, while satisfying a set of constraints. Although traditionally used in oil-refineries and process control [2], with the availability of faster computers, MPC has found widespread popularity in autonomous driving and robotic control [3]. In order to account for uncertain disturbances (typically encountered in real-life), robust MPC techniques have been proposed which add additional constraints by setting conservative bounds on disturbances during the design phase [4]. However, solving the system dynamics accurately in presence of additional constraints within the required time-budget forms a bottleneck, in high-speed applications such as agile drone navigation, even with today's hardware [5].

Artificial Intelligence (AI) / Machine learning (ML) based data-driven methods have been put forward, which perform system identification [6] by fitting kernels obtained through regression methods such as Gaussian processes [7] or neural networks [8]. These approaches if trained well, relaxes the computational demand during inference, by replacing analytical models with simple kernels [9], [10]. However, the purely data-driven AI methods lack explainability [11] and may require huge training data, with carefully annotated labels, incurring non-trivial training overheads. Physics-informed neural network (PINN) [12] introduced by Raissi et. al. approximated system of ordinary differential equations (ODEs) using a neural network. PINNs have emerged as a promising paradigm in the field of numerical optimization [13]. The residual of the ODEs are fitted using data to reduce the error using autograd – an automatic differentiation tool [14], available in standard neural-network software frameworks. Exploiting this PINN property, the main goal of this work is to perform system identification in context of MPC, via a lightweight neural network with low training overhead.

We propose *RAMP-Net* – a robust adaptive MPC via PINNs and perform trajectory tracking for a quadrotor in presence of uncertain dynamic disturbances. The PINN is trained partly from simple ODEs and partly from data. The ODEs represent the ideal system dynamics of a quadrotor in absence of uncertainties/disturbances. The data obtained through real-life-like simulated environments (with noises and disturbances) enables the proposed network to adapt to similar disturbances if encountered during inference. By training from sample sources (called collocation points [12]), whose target labels are obtained through analytical symbolic functions, we are able to infuse system knowledge in the training data. Having access to such analytical functions during training enforces desired system behavior, while also making the model partially interpretable. The main contributions of this work are as follows:

- We formulate the ideal system dynamics of a quadrotor to fit the residual dynamics as a physics loss and use a data loss to capture additional dynamics unaccounted during mathematical modelling (Section III).
- We train a PINN using the composite loss (sum of the above mentioned loss functions) to approximate the non-linear dynamics of a quadrotor to propose RAMP-Net – a robust adaptive MPC via PINNs (Section IV).
- We perform trajectory tracking of a Hummingbird quadrotor in the Gazebo simulation environment to obtain $\sim 60\%$ lesser tracking error compared to a SOTA regression-based method along with $\sim 11\%$ faster convergence. We report significant reduction in tracking

Authorized licensed use limited to: Netaji Subhas University of Technology New Delhi. Downloaded on April 04,2026 at 10:57:21 UTC from IEEE Xplore. Restrictions apply.

error for various speeds ($0.5-12.5$ m/s) w.r.t two SOTA regression based MPC methods [7], [15] (Section V).

II. Related Work

We consider trajectory tracking in the face of uncertain dynamic disturbances. Research endeavours in the past to achieve this are briefly discussed.

Deep Reinforcement Learning based Neural methods: Deep Reinforcement Learning (RL) [16], [17] approaches assume the underlying control problem to be a Markov Decision Process (MDP), and uses functional approximation to learn the optimal policy to perform sequential decision making under uncertainty. The authors of [18] have combined RL with adversarial learning [19]. The robust optimization problem is addressed using an *actor-critic* setup, where an agent (actor) learns a policy to control the system and another agent (critic) learns a separate policy to destabilize the system. The work in [20] extends [18] using an ensemble of Deep Q-Networks [21] with the actor being risk-aware and the critic being risk-seeking. Neural-MPC [22] uses Deep RL frameworks within an MPC pipeline, and High-MPC [23] exploits RL to learn high level policies from low-level MPC controllers. However, many RL approaches suffer from the sample-inefficiency with lots of training cycles and steady convergence is still a challenge in complex scenarios or in rapidly evolving environments [24]. Moreover, absence of analytical/symbolic priors results in lack of explainibility [11], making these methods tractable only for simple setups [25].

Data agnostic model-based Analytical/Symbolic methods: Tube-MPC [4] is a typical robust MPC method, which uses a nominal dynamics model and sets conservative bounds of disturbances (called *tube*) on the state variables, to obtain robust behaviour. The conservative uncertainty/disturbance states guarantee stability in the worst-case scenario, however at the expense of increased "hardness", as more constraint satisfactions are required. This problem exacerbates in performance-critical high-speed applications having strict time-budgets. To reduce the conservatism of robust controllers, adaptive MPC techniques [26], [27] consider parametric uncertainties over state variables. Such techniques either use functional analysis methods to guarantee closed-loop stability or adapts the controller parameters to mimic a reference model. However, such methods are limited to tackle only parametric uncertainties and tend to overfit to the analytical reference models, a phenomenon known as *model drift*. Hence, model-based adaptive MPC does not guarantee optimal convergence to true parameters.

Regression based system identification methods for MPC: We are interested in data-driven control methods in the context of MPC. To achieve this, we consider system identification [6], where the analytical model is improved using data-driven regression methods. This has recently inspired researchers to integrate machine learning and MPC [28]. To solve this, [7] uses Bayesian tools such as Gaussian processes to perform regression. On the other hand, [15] recently employed a technique called Neural-ODE [29] to correct modelled dynamics using neural networks. Our proposed method is along similar direction, where we use PINNs to formulate a composite loss function, with the aim to design a robust adaptive MPC framework, which we now present.

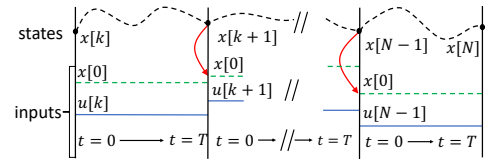

Fig. 1. Moving Horizon Illustration. Best viewed in color.

III. Proposed Approach

A. *Dynamical System formulation*

Let us consider a dynamical system of the form

$$\dot{\boldsymbol{x}}(t) = f(\boldsymbol{x}(t), \boldsymbol{u}(t)) \tag{1}$$

on the time interval $\mathbb{T} \in \mathbb{R}$, where $\boldsymbol{x} : \mathbb{T} \mapsto \mathcal{X} \in \mathbb{R}^n$ represent the state variables and $\boldsymbol{u} : \mathbb{T} \mapsto \mathcal{U} \in \mathbb{R}^m$ represent the control variables. In this study, we consider a quadrotor, with $n = 13$ states and $m = 4$ control variables. At $t = 0$, if $x(t) = x[0]$, then we are interested in solving an initial-value problem (IVP) over time-interval $\mathbb{T}$. According to [30], if f is Lipschitz-continuous with respect to the state, then the IVP has a unique solution for each $\boldsymbol{u} \in L^\infty(\mathbb{T}, \mathcal{U})$. From Eqn. (1), we can write

$$\int_{t=k}^{t=k+1} d\boldsymbol{x}(t) = \int_{t=0}^{t=T} f(\boldsymbol{x}(t), \boldsymbol{u}(t))dt \tag{2a}$$

$$\implies \boldsymbol{x}[k+1] = \boldsymbol{x}[k] + \int_{t=0}^{t=T} f(\boldsymbol{x}(t), \boldsymbol{u}(t))dt \tag{2b}$$

$$\implies \boldsymbol{x}[k+i+1] = \phi(T, \boldsymbol{x}[k+i], \boldsymbol{u}[k+i]) \tag{2c}$$

$$\forall i = \{0, 1, ..., N-1, N\} \tag{2d}$$

Figure 1 illustrates the Multiple-Shooting Moving Horizon scheme [31] of MPC, which we adopt. T represents the time-interval. The final state $\boldsymbol{x}[k+i]$ of the previous interval is fed as the initial state $\boldsymbol{x}[0]$ of the next interval to obtain $\boldsymbol{x}[k+i+1]$ by solving an IVP for each interval T. This enables a time-discretized implementation of a digital controller. We make a zero-hold assumption for the control variable, i.e, $\dot{\boldsymbol{u}}[k] \equiv 0$, meaning the control signals are constant and discrete within a moving horizon interval (from $t = 0$ to $t = T$). ϕ represents a regressor (model-based or data-based or both) to be fitted.

B. *MPC problem formulation*

Given a reference trajectory $\boldsymbol{x}_k^{ref}$, and a control system as described in Eqn. (2), we want the state vector ($\boldsymbol{x}_k$) of a quadrotor to follow $\boldsymbol{x}_k^{ref}$ as closely as possible. We use a quadratic cost $J \in \mathbb{R}$, defined as follows:

$$J(\boldsymbol{x}_k, \boldsymbol{u}_k) = \sum_{i=k}^{k+T-1} (||\boldsymbol{x}_i^{ref} - \boldsymbol{x}_i^{pred}||_{\boldsymbol{Q}}^2 + ||\boldsymbol{u}_i||_{\boldsymbol{R}}^2) \tag{3}$$

Authorized licensed use limited to: Netaji Subhas University of Technology New Delhi. Downloaded on April 04,2026 at 10:57:21 UTC from IEEE Xplore. Restrictions apply.

where $||\boldsymbol{x}||_{\boldsymbol{Q}} = \sqrt{\boldsymbol{x}^T \boldsymbol{Q} \boldsymbol{x}} : \mathbb{R}^n \mapsto \mathbb{R}$, $||\boldsymbol{u}||_{\boldsymbol{R}} = \sqrt{\boldsymbol{u}^T \boldsymbol{R} \boldsymbol{u}} : \mathbb{R}^m \mapsto \mathbb{R}$ are weighted semi-norms $\forall\ \boldsymbol{Q} \in \mathbb{R}^{n \times n}$, $\boldsymbol{R} \in \mathbb{R}^{m \times m}$ being positive semi-definite. The MPC problem then involves iteratively solving the following in real-time:

$$\underset{\boldsymbol{u}_k}{\operatorname{argmin}}\, J(\boldsymbol{x}_k, \boldsymbol{u}_k) \tag{4a}$$

$$\text{s.t.} \quad \boldsymbol{x}[k+1] = \phi(T, \boldsymbol{x}[k], \boldsymbol{u}[k]) \tag{4b}$$

$$\forall\ \boldsymbol{x}[k] \in \mathcal{X},\ \ \boldsymbol{u}[k] \in \mathcal{U} \tag{4c}$$

$$\forall\ k \in \{0, 1, ..., N, N-1\} \tag{4d}$$

By abuse of notation, $\boldsymbol{x}_k = \boldsymbol{x}[k]$, and $\boldsymbol{u}_k = \boldsymbol{u}[k]$. For each interval, the optimal control signals $\boldsymbol{u}_k$ are obtained by solving Eqn. (4). Traditionally, model-based MPC employs non-linear programming methods such as interior-point-optimization (IP-OPT) [32] and uses numerical integration schemes such as Runge-Kutta methods. For high speed applications, evaluating ϕ can be challenging, specially if additional constraints are imposed for robust tracking, by setting conservative bounds. For complex environments, ϕ can be hard to model accurately. In this work, we use a PINN to numerically evaluate ϕ, enabling rapid system identification for a robust adaptive MPC. We now give a brief background on PINNs and explain how it can be modified to perform system identification of a quadrotor subjected to uncertain dynamic disturbances.

C. *Physics-informed Neural Networks*

Physics-informed Neural Networks (PINNs) [12] was introduced in 2019 by Raissi et al. . It solved differential equations by adding the differential equation to the loss function itself, as a residual. If we consider Eqn. (1), we can formulate a residual physics loss as follows:

$$\mathcal{L}_p = MSE(\dot{\boldsymbol{x}}(t), f(\boldsymbol{x}(t), \boldsymbol{u}(t))) \tag{5}$$

MSE stands for the mean square error. The original paper only considered equations with state variables $\boldsymbol{x}$. However to solve an MPC, we require addition of control variables $\boldsymbol{u}$. In this work, we add provision for using the control variable $\boldsymbol{u}$, and a time variable t, to be fed to the neural network, as separate signals, similar to [33]. Substituting the continuous variables, with ϕ from Eqn. (4), in this work, we arrive at the corresponding physics loss, as shown:

$$\mathcal{L}_p = MSE(\dot{\phi}(T, \boldsymbol{x}_k, \boldsymbol{u}_k), f(\boldsymbol{x}_k, \boldsymbol{u}_k)) \tag{6a}$$

$$\implies \mathcal{L}_p = MSE(\dot{\phi_T}(\boldsymbol{x}_k, \boldsymbol{u}_k; \theta), f(\boldsymbol{x}_k, \boldsymbol{u}_k)) \tag{6b}$$

$$\implies \mathcal{L}_p = \frac{1}{|\mathcal{P}|} \sum_{k=1}^{|\mathcal{P}|} ||\dot{\phi_T}(\boldsymbol{x}_k, \boldsymbol{u}_k; \theta) - f(\boldsymbol{x}_k, \boldsymbol{u}_k)||^2 \tag{6c}$$

$$\forall\ \{\boldsymbol{x}_k, \boldsymbol{u}_k\} \in \mathcal{P} \tag{6d}$$

We use a neural network parametrized by θ to learn ϕ_T and treat T as the network parameter, dropping it as a function argument. Using autograd [14] – an automatic differentiation tool readily available in standard neural network packages, the numerical derivative $\dot{\phi_T}$ can be easily evaluated in one forward propagation. $\{\boldsymbol{x}_k, \boldsymbol{u}_k\} \in \mathcal{P}$ are called *collocation points* [12], where $\mathcal{P}$ is the physics dataset. In the physics loss (Eqn. 6), instead of a labelled target, the symbolic prior applied on the collocation points ($f(\boldsymbol{x}, \boldsymbol{u})$) is used, imposing a constraint on the data loss (Eqn. 7). This constrains/regularizes the neural network to obey the dynamics of a quadrotor modelled in f.

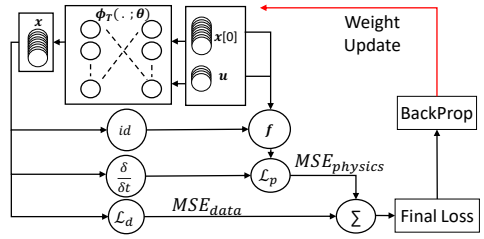

Fig. 2. PINN Loss = Physics Loss + Data Loss. id implies identity operation.

Figure 2 illustrates the implemented PINN loss evaluation. A simple Multilayer Perceptron (MLP) is trained. The control variable $\boldsymbol{u}$ along with the initial values $\boldsymbol{x}[0]$ (last measurement from the previous horizon) are fed as inputs. The PINN MLP output is passed through f (the ideal dynamics) and $\frac{\delta}{\delta t}$ (the *autograd* function) to obtain the physics loss. Furthermore, we collect recorded observations to prepare a dataset $\mathcal{D}$ which to fit a data loss as follows:

$$\mathcal{L}_d = MSE(\phi_T(\boldsymbol{x}_i, \boldsymbol{u}_i; \theta), \boldsymbol{y}_i) \tag{7a}$$

$$\implies \mathcal{L}_d = \frac{1}{|\mathcal{D}|} \sum_{i=1}^{|\mathcal{D}|} ||\phi_T(\boldsymbol{x}_i, \boldsymbol{u}_i; \theta) - \boldsymbol{y}_i)||^2 \tag{7b}$$

$$\forall\ \{(\boldsymbol{x}_i, \boldsymbol{u}_i), \boldsymbol{y}_i\} \in \mathcal{D} \tag{7c}$$

The dataset $\mathcal{D}$ consists of uncertain dynamic disturbances, obtained through noise injections in a simulated physics engine. The data samples $\{\boldsymbol{x}_i, \boldsymbol{u}_i\} \in \mathcal{D}$ represent the state and control input measurements obtained through odometry, and $\{\boldsymbol{y}_i\}$, the corresponding groundtruth label of the quadrotor state derivatives. The physics loss and data loss are added together to obtain the final composite PINN loss ($\mathcal{L}_p + \mathcal{L}_d$), which is backpropagated to perform the weight updates. Later, we show quantitatively the relative impact of varying $|\mathcal{P}|$ and $|\mathcal{D}|$ (Section V-C).

IV. System Design

Figure 3 illustrates the logical view of the proposed RAMP-Net architecture. A switch S_t is used to toggle between the training ($S_t =$ **ON**) and inference ($\bar{S}_t =$ **ON**). We summarize the quadrotor nominal dynamics model similar to [34], [35] and subsequently describe how we achieve robust behavior to tackle the issue of uncertain dynamics in context of MPC, when S_t is set to **ON**.

A. *Quadrotor Nominal Dynamics*

Consider a six degrees-of-freedom quadrotor with mass m and diagonal moment of inertia $\boldsymbol{J} = \text{diag}(J_x, J_y, J_z)$. We define $\boldsymbol{x} = [\boldsymbol{p}, \boldsymbol{q}, \boldsymbol{v}, \boldsymbol{\omega}_B]\ \forall\ \boldsymbol{x} \in \mathbb{R}^{13}$ as the quadrotor state variables. $\boldsymbol{p} = [x, y, z]^T\ \forall\ \boldsymbol{p} \in \mathbb{R}^3$ are the quadrotor position coordinates in the world frame. We use the unit quaternions

Authorized licensed use limited to: Netaji Subhas University of Technology New Delhi. Downloaded on April 04,2026 at 10:57:21 UTC from IEEE Xplore. Restrictions apply.

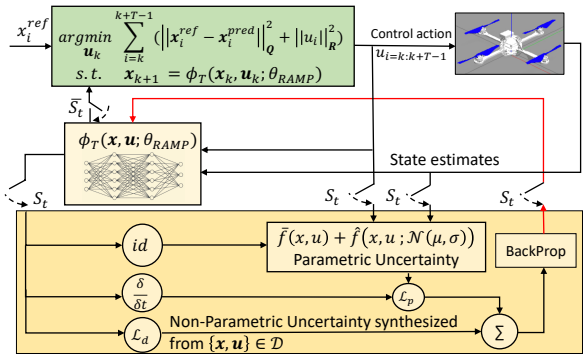

Fig. 3. RAMP-Net Architecture. Best viewed in color.

$\boldsymbol{q} \in \mathbb{R}^4 = [q_w, q_x, q_y, q_z]^T$ such that $||\boldsymbol{q}|| = 1$ to represent the quadrotor attitude, also in the world frame. $\boldsymbol{v} \in \mathbb{R}^3$ are the linear velocities, i.e. $\boldsymbol{v} = \dot{\boldsymbol{p}}$ in the world frame, and $\boldsymbol{\omega}_B \in \mathbb{R}^3$ denotes the angular velocities along XYZ axes in the body frame. We model the quadrotor thrusts $T_i \ \forall \ i \in \{0, 1, 2, 3\}$ as the control input signals $\boldsymbol{u} \in \mathbb{R}^4$. The quadrotor nominal dynamics is modelled as follows:

$$\dot{\boldsymbol{x}} = \begin{bmatrix} \dot{\boldsymbol{p}} \\ \dot{\boldsymbol{q}} \\ \dot{\boldsymbol{v}} \\ \dot{\boldsymbol{\omega}}_B \end{bmatrix} = \bar{f}(\boldsymbol{x}, \boldsymbol{u}) = \begin{bmatrix} \boldsymbol{v} \\ \boldsymbol{q} \cdot \begin{bmatrix} 0 \\ \boldsymbol{\omega}_B/2 \end{bmatrix} \\ \frac{1}{m}\boldsymbol{q} \odot \boldsymbol{T}_B + \boldsymbol{g} \\ \boldsymbol{J}^{-1}(\boldsymbol{\tau}_B - \boldsymbol{\omega}_B \times \boldsymbol{J}\boldsymbol{\omega}_B) \end{bmatrix} \tag{8a}$$

$$\boldsymbol{g} = \begin{bmatrix} 0 \\ 0 \\ -9.8 \end{bmatrix}, \boldsymbol{T}_B = \begin{bmatrix} 0 \\ 0 \\ \sum T_i \end{bmatrix} \tag{8b}$$

$$\boldsymbol{\tau}_B = \begin{bmatrix} L(-T_0 - T_1 + T_2 + T_3) \\ L(-T_0 + T_1 + T_2 - T_3) \\ k_{drag}(-T_0 + T_1 - T_2 + T_3) \end{bmatrix} \tag{8c}$$

where $\boldsymbol{T}_B$ is the collective thrust acting upward, $\boldsymbol{\tau}_B$ is the body torque, k_{drag} is the drag constant and L is the quadrotor arm length in $\times$ configuration. $\boldsymbol{q} \odot \boldsymbol{T}_B$ indicates the rotation of vector body torque by the quadrotor attitude, i.e., $\boldsymbol{q} \odot \boldsymbol{T}_B = \boldsymbol{q}\boldsymbol{T}_B\bar{\boldsymbol{q}}$, where $\bar{\boldsymbol{q}}$ is the conjugate quaternion.

B. Robustness through infusing Parametric Uncertainty

We modify the system dynamics governed by function f (see Eqn. (1)) as follows:

$$f(\boldsymbol{x}, \boldsymbol{u}) = \bar{f}(\boldsymbol{x}, \boldsymbol{u}) + \hat{f}(\boldsymbol{x}, \boldsymbol{u}) \tag{9}$$

where $\bar{f}(\boldsymbol{x}, \boldsymbol{u})$ represents the nominal dynamics in Eqn. (8) and $\hat{f}(\boldsymbol{x}, \boldsymbol{u})$ represents the additive parametric uncertainty representing deviations from the nominal quadrotor state variables. Such deviations can be sampled from standard distributions, such as a normal distribution parameterized by $\mathcal{N}(\mu, \sigma)$, μ and σ being the mean and standard deviations respectively (see Section V-A). This additive term is included in the symbolic prior, influencing the physics loss $\mathcal{L}_p$ as:

$$\mathcal{L}_p = \frac{1}{|\mathcal{P}|}\sum_{k=1}^{|\mathcal{P}|} ||\dot{\phi}_T(\boldsymbol{x}_k, \boldsymbol{u}_k; \theta_{RAMP}) - (\bar{f} + \hat{f})||^2 \tag{10a}$$

$$\forall \ \bar{f} = \bar{f}(\boldsymbol{x}_k, \boldsymbol{u}_k) \tag{10b}$$

$$\forall \ \hat{f} = \hat{f}(\boldsymbol{x}_k, \boldsymbol{u}_k; \mathcal{N}(\mu, \sigma)) \tag{10c}$$

C. Adaptation to Residual Non-Parametric Uncertainty

In addition to analytical modelling (which includes the physics loss), we utilize data-driven methods to obtain more accurate dynamics. External environment conditions such as winds, disturbances and frictional effects on rotor inertia cannot be parameterized using quadrotor states. We add a non-parametric term $\tilde{f}$ in Eqn. (9) as follows:

$$f_{true}(\boldsymbol{x}, \boldsymbol{u}) = \bar{f}(\boldsymbol{x}, \boldsymbol{u}) + \hat{f}(\boldsymbol{x}, \boldsymbol{u}) + \tilde{f} \tag{11}$$

where $\tilde{f} : \mathbb{T} \mapsto \mathcal{X} \in \mathbb{R}^n$. We fly the quadrotor in a simulated environment which injects various noises and disturbances. Specifically, we inject zero-mean Gaussian noises with 1 standard deviation on the rotor thrusts, a 2nd order polynomial aerodynamic drag effect and add asymmetric motor voltage noises in the simulated environment while preparing the dataset $\mathcal{D}$. From Eqn. (4b), using shorthand, we have

$$\boldsymbol{x}_{k+1} = \phi_T(\boldsymbol{x}_k, \boldsymbol{u}_k; \theta_{RAMP}) \tag{12a}$$

$$\implies \boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \int_{t=0}^{t=T} f_{true}(\boldsymbol{x}, \boldsymbol{u})dt \tag{12b}$$

$$= \boldsymbol{x}_k + \int_{t=0}^{t=T} (\bar{f} + \hat{f})(\boldsymbol{x}, \boldsymbol{u})dt + \tilde{f} \tag{12c}$$

$\tilde{f}$ is hence synthesized from the dataset $\mathcal{D}$ affecting the data loss $\mathcal{L}_d$, which is rewritten as:

$$\mathcal{L}_d = \frac{1}{|\mathcal{D}|}\sum_{k=1}^{|\mathcal{D}|} ||\boldsymbol{x}_{k+1} - \boldsymbol{x}_k - \int_{t=0}^{t=T} (\bar{f} + \hat{f})(\boldsymbol{x}_k, \boldsymbol{u}_k)dt||^2 \tag{13}$$

$\mathcal{D} = [((\boldsymbol{x}_1, \boldsymbol{u}_1), \boldsymbol{y}_1), ((\boldsymbol{x}_2, \boldsymbol{u}_2), \boldsymbol{y}_2), ..., ((\boldsymbol{x}_{|\mathcal{D}|}, \boldsymbol{u}_{|\mathcal{D}|}), \boldsymbol{y}_{|\mathcal{D}|})]^T$ logged at times $[t_1, t_2, ..., t_{|\mathcal{D}|}]$, where $\boldsymbol{y}_k$ is the integrand in Eqn. (13) $\forall \ k \in \{1, 2, ..., |\mathcal{D}|\}$.

We add the two losses $\mathcal{L}_p$ and $\mathcal{L}_d$ to train our PINN θ_{RAMP} in order to identify our perceived dynamics in inference mode (when $\bar{S}_t$ is set to **ON**) as:

$$\boldsymbol{x}_{k+1} = \phi_T(\boldsymbol{x}_k, \boldsymbol{u}_k; \theta_{RAMP}). \tag{14}$$

V. Results

A. Methodology

We implemented the PINN using Tensorflow [36], following the approach in [37]. The symbolic nominal dynamics of the quadrotor were implemented in ACADOS [38] which wraps around the non-linear optimization toolkit CasAdi [39]. We used a 4 layer MLP with 128 neurons in each layer as our PINN architecture, and trained for 2000 epochs using early stopping [40] with a patience of 500 epochs. We used a learning rate of 1. For parametric uncertainty, we used a zero mean normal distribution with unit standard deviation ($\mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma})$), where $\boldsymbol{\Sigma}$ is a constant unit diagonal covariance matrix. We used the entire dataset as a single batch using the memory-efficient quasi-newton L-BFGS optimizer [41] following [12], [13]. We used $5k$ sample points as $|\mathcal{P}| + |\mathcal{D}|$. The weighted coefficients for $\mathcal{L}_p$ and $\mathcal{L}_d$ were set

Authorized licensed use limited to: Netaji Subhas University of Technology New Delhi. Downloaded on April 04,2026 at 10:57:21 UTC from IEEE Xplore. Restrictions apply.

to unity. The PINN based RAMP-Net framework was tested on closed-loop trajectory tracking experiments in the Gazebo [42] environment using the AscTec Hummingbird quadrotor model from the RotorS framework [43]. Table I presents the implementation details.

TABLE I
RAMP-NET IMPLEMENTATION DETAILS

Quadrotor Property	Value
Mass (m)	$(0.68 + 4 \times 0.009)$ kg
Arm Length (L)	0.17 m
Drag Constant (k_{drag})	$8.06e - 05$
Max Rotor Speed	838 rad/s
(J_x, J_y, J_z)	$(0.007, 0.007, 0.012)\ kgm^2$
MPC Settings	**Value**
Time Horizon (T)	1 sec
Publish Frequency	500 Hz
$\boldsymbol{Q}[(0,0):(2,2)]$	0.5
$\boldsymbol{Q}[(3,3):(6,6)]$	0.1
$\boldsymbol{Q}[(7,7):(9,9)]$	0.05
$\boldsymbol{Q}[(10,10):(12,12)]$	0.01
$\boldsymbol{R}$	diag(0.1,0.1,0.1,0.1)

B. *Comparitive Schemes*

- **KNODE-MPC [15]**: This work uses Neural ODEs [29] to learn the residual dynamics which is added to a nominal dynamics model.
- **GP-MPC**: This scheme employs a Gaussian Process to learn the residual dynamics as a posterior probability distribution, given a prior nominal dynamics.
- **Ideal**: This considers the nominal dynamics only as the ideal case, but with perfect oracle of the uncertainties applied as the residual dynamics.
- **Nominal**: This considers the nominal dynamics only without any data-driven correction.
- **PID**: This assumes an oracle of the uncertainties and considers a perfectly fine-tuned adaptive PID controller.

C. *Impact of data on training performance*

We evaluate the training performance in terms of 1) tracking error and 2) total training time. We model the tracking error as a similarity distance between the predicted and desired trajectories using the dynamic time warping (DTW) algorithm [44] implemented using [45], using a metric called data-skewness ($|\mathcal{D}|/|\mathcal{P}|$) which is the ratio of number of regular data points w.r.t the number of collocation points.

Figure 4 illustrates the impact of varying data-skewness. For KNODE-MPC, this translates to the sample complexity as discussed in the paper, where the nominal dynamics is disjoint from the neural network. We swipe from a highly skewed dataset ($1/16$) to zero skew ($1/1$), with intermediate degrees of skewness ($1/8$, $1/4$, $1/2$). We plot the % DTW errors normalized w.r.t. the nominal baselines for RAMP-Net and KNODE-MPC for circular trajectories of radius $3m$ and $4m$ along with the training times in seconds. The results obtained are for maximum radial velocity of $1m/s$ at a steady height of $\sim 1m$. For $|\mathcal{D}|/|\mathcal{P}| = 1/16$, we observe that 3 out of 4 % DTW errors exceed 100% over nominal. This indicates that with extreme skewness (very less data), the

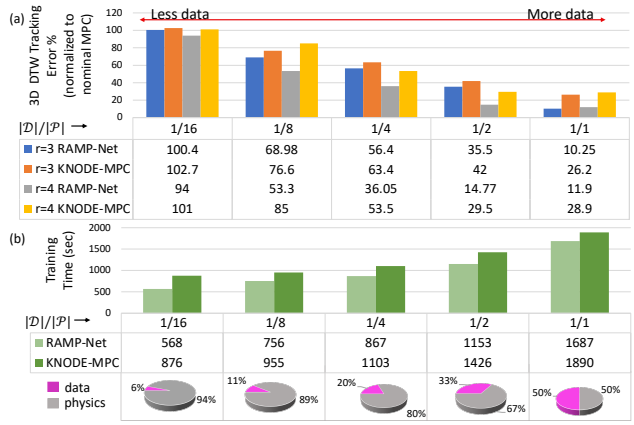

Fig. 4. Role of data vs physics points on training performance for 2k epochs (a) 3D DTW Tracking Error (b) Training Time (sec). Best viewed in color.

PINNs do not acquire sufficient expressive power for system identification. With reduction in skewness, we observe:

1) Errors reduce for both methods, and 2) The error reduction in RAMP-Net is greater than KNODE-MPC with decreasing skewness. For zero skewness, we report $\sim 61\%$ and $\sim 59\%$ lesser errors for circles with radii $3m$ and $4m$ respectively, compared to KNODE-MPC, with $\sim 11\%$ faster convergence, on the entire training set. Increasing the number of data-points beyond zero-skew (i.e. $|\mathcal{D}|/|\mathcal{P}| > 1$) can overfit to the odometry data and also increases the training overhead. Hence, for subsequent evaluations, we chose a balanced dataset with no skew (i.e. $|\mathcal{D}|/|\mathcal{P}| = 1$).

D. *Comparison with SOTA Regression based Methods*

We compare our work with two other regression based system identification methods for MPC – KNODE-MPC, and GP-MPC. Figure 5 presents the % DTW errors of the three methods for circle and lemniscate trajectories, normalized w.r.t nominal MPC. Though the methods are trained only on circles with radii $3m$ and $4m$, the evaluation is performed for lemniscate trajectories as well, to showcase the generalization capability of data-driven techniques aiding MPC. The horizontal axes in Figure 5 represent different trajectory radii, while the vertical axes represent the top radial speed. We considered stable hovering at $1m$ for the experiments.

The errors are more in most cases for lemniscate compared to circle trajectory. This is due to the lemniscate shape being more complex than a simple circle, places higher demands on the motor control (more yawing is needed) in presence of environment disturbances. For RAMP-Net, we observe higher variation with increasing speed across different radii (8.45, 6.24, 4.98 standard deviations for circle and 11.65, 10.17, 4.42 standard deviations for lemniscate for RAMP-Net, KNODE-MPC and GP-MPC respectively). However, RAMP-Net outperforms both KNODE-MPC and GP-MPC in terms of % DTW errors normalized to nominal MPC (46.07%, 53.63%, and 78% for circle and 47.41%, 80.65%, and 92.79% for lemniscate for RAMP-Net, KNODE-MPC and GP-MPC respectively, on average). For speeds ranging from 0.5 m/s to 1.75 m/s, RAMP-Net outperforms KNODE-

Authorized licensed use limited to: Netaji Subhas University of Technology New Delhi. Downloaded on April 04,2026 at 10:57:21 UTC from IEEE Xplore. Restrictions apply.

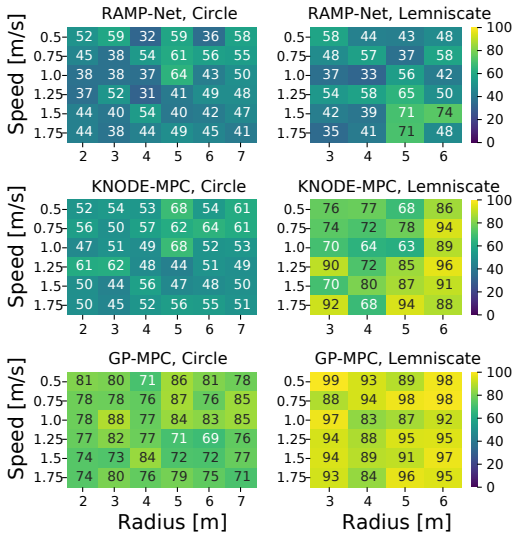

Fig. 5. Heatmap of DTW errors normalized to nominal. Lower is better. Best viewed in color.

MPC and GP-MPC by $7.8\% - 43.2\%$, and by $8.04\% - 61.5\%$ respectively, on average, in terms of DTW error.

E. Robustness Analysis for higher flight speeds

Figure 6 presents the root-mean-square errors (RMSE) of all the comparative schemes for radius = 3m, height = 1m. Table II reports the corresponding relative increase in RMSE from ideal, i.e., $|x - ideal|/ideal$, where x is a comparative scheme. We vary the maximum radial speed as $2.5, 5.0, 7.5, 10.0, 12.5$ m/s. The environment consists of dynamic disturbances along with wind and translational drag effects on the rotors. For the ideal case, the dynamic disturbances are perfectly countered, resulting in the least tracking error. However, with increasing maximum speed, the available time budget reduces, increasing the tracking error. For nominal, there is no corrective measure to counter the disturbances, causing the highest increase. The PID control considers the best tuned gains, however the PID errors are still considerably high ($\sim 27\times$ from ideal). We observe that the errors for GP-MPC, and KNODE-MPC are uncorrelated with maximum radial speed (within $\sim 15\times$, and $\sim 13\times$ from ideal). RAMP-Net offers the best adaptation, with least tracking error (within $\sim 10\times$ from ideal), with sub-linear increase (similar to the GP-MPC and KNODE-MPC), with increasing speed (disturbances), indicating robust tracking.

TABLE II
RELATIVE AVERAGE RMSE INCREASE NORMALIZED TO IDEAL. LOWER IS BETTER.

	Nominal	PID	GP-MPC	KNODE-MPC	RAMP-Net
Random	4.61	3.59	1.49	1.38	**1.05**
Circle	44.23	29.81	13.35	11.02	**8.43**
Lemniscate	51.45	46.67	30.01	26.97	**20.34**
Average	33.43	26.67	14.95	13.12	**9.94**

F. Latency comparison with standard integrators

Table III presents the execution run-times obtained using RAMP-Net and other standard integration methods like

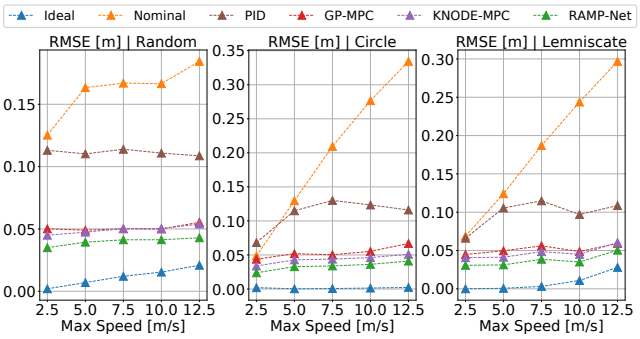

Fig. 6. RMSE Errors. Lower is better. Best viewed in color.

Eulers and Runge-Kutta (RK4, RK45) methods, used in MPC without data-driven regression. We report an order of magnitude lower latency. The times reported for the RAMP-Net PINNs are the wall-clock times observed when running Tensorflow on an NVIDIA GeForce RTX 2080 Ti GPU with 4 cards, 1 GB memory and a clock of 300 MHz. We expect higher speedup with dedicated accelerators and lower-level software routines such as BLAS in C/C++.

TABLE III
EXECUTION TIME FOR 1 FORWARD PROPAGATION

	RAMP-Net	Euler	RK4	RK45
Mean (sec)	4.14e-04	5.25e-03	2.6e-03	9.4e-03
Median (sec)	2.67e-04	5.09e-03	2.52e-03	5.4e-03

VI. CONCLUSION

Pure model based robust MPC techniques suffer performance degradation when subjected to uncertain dynamic disturbances (Nominal case in Figure 6). To that effect, we proposed RAMP-Net – a robust adaptive MPC framework which uses a neural network that embeds the system (in our case it is a quadrotor) dynamics directly in the neural network loss forming a composite loss function. Experiments performed on a Hummingbird quadrotor in the Gazebo simulation environment reveal that our proposed method results in $\sim 60\%$ lesser tracking error while training compared to a SOTA regression based method [15] along with $\sim 11\%$ faster convergence. We report significant reduction in tracking error for various speeds ($0.5 - 12.5$ m/s) compared to two SOTA regression based MPC methods [7], [15] and three standard controllers, with faster dynamics integration compared to traditional numerical integration methods. The results establish the effectiveness of incorporating physics-based AI models for solving optimal control problems in noisy settings. This potentially should allow researchers to combine first-principle models with neural networks to better identify real-world dynamical systems.

VII. ACKNOWLEDGEMENT

This work was supported in part by the Center for Brain-inspired Computing (C-BRIC), a DARPA sponsored JUMP center, the Semiconductor Research Corporation (SRC), the National Science Foundation, the DoD Vannevar Bush Fellowship, and IARPA MicroE4AI.

Authorized licensed use limited to: Netaji Subhas University of Technology New Delhi. Downloaded on April 04,2026 at 10:57:21 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] E. F. Camacho and C. B. Alba, *Model predictive control*. Springer science & business media, 2013.

[2] U. Yüzgeç, A. Palazoglu, and J. A. Romagnoli, "Refinery scheduling of crude oil unloading, storage and processing using a model predictive control strategy," *Computers & Chemical Engineering*, vol. 34, no. 10, pp. 1671–1686, 2010.

[3] G. P. Incremona, A. Ferrara, and L. Magni, "Mpc for robot manipulators with integral sliding modes generation," *IEEE/ASME Transactions on Mechatronics*, vol. 22, no. 3, pp. 1299–1307, 2017.

[4] D. Q. Mayne, M. M. Seron, and S. Raković, "Robust model predictive control of constrained linear systems with bounded disturbances," *Autom.*, vol. 41, pp. 219–224, 2005.

[5] J. Oravec, Y. Jiang, B. Houska, and M. Kvasnica, "Parallel explicit mpc for hardware with limited memory," *IFAC-PapersOnLine*, vol. 50, no. 1, pp. 3301–3306, 2017.

[6] S. L. Brunton, J. L. Proctor, and J. N. Kutz, "Sparse identification of nonlinear dynamics with control (sindyc)," *IFAC-PapersOnLine*, vol. 49, no. 18, pp. 710–715, 2016.

[7] G. Torrente, E. Kaufmann, P. Föhn, and D. Scaramuzza, "Data-driven mpc for quadrotors," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 3769–3776, 2021.

[8] K. Patan, "Neural network-based model predictive control: Fault tolerance and stability," *IEEE Transactions on Control Systems Technology*, vol. 23, no. 3, pp. 1147–1155, 2014.

[9] S. Sanyal and K. Roy, "Neuro-ising: Accelerating large-scale traveling salesman problems via graph neural network guided localized ising solvers," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 12, pp. 5408–5420, 2022.

[10] S. Sanyal, A. Ankit, C. M. Vineyard, and K. Roy, "Energy-efficient target recognition using reram crossbars for enabling on-device intelligence," in *2020 IEEE Workshop on Signal Processing Systems (SiPS)*, 2020, pp. 1–6.

[11] J. M. Benítez, J. L. Castro, and I. Requena, "Are artificial neural networks black boxes?" *IEEE Transactions on neural networks*, vol. 8, no. 5, pp. 1156–1164, 1997.

[12] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational physics*, vol. 378, pp. 686–707, 2019.

[13] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang, "Physics-informed machine learning," *Nature Reviews Physics*, vol. 3, no. 6, pp. 422–440, 2021.

[14] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, "Automatic differentiation in machine learning: a survey," *Journal of Marchine Learning Research*, vol. 18, pp. 1–43, 2018.

[15] K. Y. Chee, T. Z. Jiahao, and M. A. Hsieh, "Knode-mpc: A knowledge-based data-driven predictive control framework for aerial robots," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 2819–2826, 2022.

[16] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath, "Deep reinforcement learning: A brief survey," *IEEE Signal Processing Magazine*, vol. 34, no. 6, pp. 26–38, 2017.

[17] M. Turchetta, A. Krause, and S. Trimpe, "Robust model-free reinforcement learning with multi-objective bayesian optimization," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 10 702–10 708.

[18] L. Pinto, J. Davidson, R. Sukthankar, and A. Gupta, "Robust adversarial reinforcement learning," in *International Conference on Machine Learning*. PMLR, 2017, pp. 2817–2826.

[19] J. Goodfellow, "Pouget-abadie, m," *Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, Y. Bengio, Generative Adversarial Networks*, 2014.

[20] X. Pan, D. Seita, Y. Gao, and J. Canny, "Risk averse robust adversarial reinforcement learning," in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 8522–8528.

[21] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, "Human-level control through deep reinforcement learning," *nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[22] T. Salzmann, E. Kaufmann, M. Pavone, D. Scaramuzza, and M. Ryll, "Neural-mpc: Deep learning model predictive control for quadrotors and agile robotic platforms," *arXiv preprint arXiv:2203.07747*, 2022.

[23] Y. Song and D. Scaramuzza, "Learning high-level policies for model predictive control," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.

[24] B. Dai, A. Shaw, L. Li, L. Xiao, N. He, Z. Liu, J. Chen, and L. Song, "Sbeed: Convergent reinforcement learning with nonlinear function approximation," in *International Conference on Machine Learning*. PMLR, 2018, pp. 1125–1134.

[25] L. Brunke, M. Greeff, A. W. Hall, Z. Yuan, S. Zhou, J. Panerati, and A. P. Schoellig, "Safe learning in robotics: From learning-based control to safe reinforcement learning," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 5, pp. 411–444, 2022.

[26] M. Bujarbaruah, X. Zhang, U. Rosolia, and F. Borrelli, "Adaptive mpc for iterative tasks," in *2018 IEEE Conference on Decision and Control (CDC)*, 2018, pp. 6322–6327.

[27] A. Dhar and S. Bhasin, "Indirect adaptive mpc for discrete-time lti systems with parametric uncertainties," *IEEE Transactions on Automatic Control*, vol. 66, no. 11, pp. 5498–5505, 2021.

[28] M. Qraitem, D. Kularatne, E. Forgoston, and M. A. Hsieh, "Bridging the gap: Machine learning to resolve improperly modeled dynamics," *Physica D: Nonlinear Phenomena*, vol. 414, p. 132736, 2020.

[29] R. T. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud, "Neural ordinary differential equations," *Advances in neural information processing systems*, vol. 31, 2018.

[30] E. Sonntag, "Mathematical control theory: Deterministic finite dimensional systems," *Springer Nature*, vol. 2, 2013.

[31] L. T. Biegler, *Nonlinear programming: concepts, algorithms, and applications to chemical processes*. SIAM, 2010.

[32] S. Wright, J. Nocedal *et al.*, "Numerical optimization," *Springer Science*, vol. 35, no. 67-68, p. 7, 1999.

[33] E. A. Antonelo, E. Camponogara, L. O. Seman, E. R. de Souza, J. P. Jordanou, and J. F. Hubner, "Physics-informed neural nets for control of dynamical systems," *arXiv preprint arXiv:2104.02556*, 2021.

[34] D. Mellinger and V. Kumar, "Minimum snap trajectory generation and control for quadrotors," in *2011 IEEE International Conference on Robotics and Automation*, 2011, pp. 2520–2525.

[35] M. Kamel *et al.*, *Model Predictive Control for Trajectory Tracking of Unmanned Aerial Vehicles Using Robot Operating System*. Springer International Publishing, 2017, vol. 2, pp. 3–39.

[36] M. Abadi, A. Agarwal *et al.*, "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015, software available from tensorflow.org. [Online]. Available: https://www.tensorflow.org/

[37] J. Nicodemus, J. Kneifl, J. Fehr, and B. Unger, "Physics-informed neural networks-based model predictive control for multi-link manipulators," 2021. [Online]. Available: https://arxiv.org/abs/2109.10793

[38] B. Houska, H. Ferreau, and M. Diehl, "ACADO Toolkit – An Open Source Framework for Automatic Control and Dynamic Optimization," *Optimal Control Applications and Methods*, vol. 32, no. 3, pp. 298–312, 2011.

[39] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "CasADi – A software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation*, vol. 11, no. 1, pp. 1–36, 2019.

[40] L. Prechelt, "Early stopping - but when?" in *Neural Networks: Tricks of the Trade, volume 1524 of LNCS, chapter 2*. Springer-Verlag, 1997, pp. 55–69.

[41] D. C. Liu and J. Nocedal, "On the limited memory bfgs method for large scale optimization," *Mathematical programming*, vol. 45, no. 1, pp. 503–528, 1989.

[42] N. Koenig and A. Howard, "Design and use paradigms for gazebo, an open-source multi-robot simulator," in *2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (IEEE Cat. No.04CH37566)*, vol. 3, 2004, pp. 2149–2154 vol.3.

[43] F. Furrer, M. Burri, M. Achtelik, and R. Siegwart, *Robot Operating System (ROS): The Complete Reference (Volume 1)*. Cham: Springer International Publishing, 2016, ch. RotorS—A Modular Gazebo MAV Simulator Framework, pp. 595–625.

[44] H. Sakoe and S. Chiba, "Dynamic programming algorithm optimization for spoken word recognition," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 26, no. 1, pp. 43–49, 1978.

[45] S. Tavenard, J. Faouzi *et al.*, "Tslearn a machine learning toolkit for time series data," *Journal of Machine Learning Research*, vol. 21, no. 118, pp. 1–6, 2020.

Authorized licensed use limited to: Netaji Subhas University of Technology New Delhi. Downloaded on April 04,2026 at 10:57:21 UTC from IEEE Xplore. Restrictions apply.